

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java — no database, just Swing and pure OOP.

● Week 8 Complete

Java JDK 21

VS Code

Swing / JPanel

Full Life + Win/Lose
Flow

WHAT WAS BUILT

Game State Management

- ✓ GameState.java — enum: MENU, PLAYING, PAUSED, GAME_OVER, WIN
- ✓ Single state field replaces the old won/gameOver booleans
- ✓ keyPressed() branches by state — ENTER/SPACE starts from MENU, P/ESC toggles pause
- ✓ paintComponent() renders a different overlay per state
- ✓ GAME_OVER and WIN screens both support a restart prompt

Collision & Life Loss

- ✓ checkGhostCollisions() now handles life loss, not just ghost-eating
- ✓ Non-frightened contact calls scoreManager.loseLife()
- ✓ resetPositions() snaps Pac-Man + all 4 ghosts back to spawn tiles
- ✓ Frightened timer is cleared on death to avoid stale frightened state
- ✓ isGameOver() transitions state to GAME_OVER only at zero lives

KEY TECHNIQUES

One Enum Replaces Two Booleans

Earlier weeks tracked won and gameOver as separate booleans that could, in theory, both be true at once. Consolidating them into a single GameState state field makes MENU, PLAYING, PAUSED, GAME_OVER, and WIN mutually exclusive by construction, and lets both keyPressed() and paintComponent() switch on the same value instead of juggling flag combinations.

Same Tile-Equality Check, Two Different Outcomes

checkGhostCollisions() reuses the row/col comparison already trusted for ghost-eating detection from Week 7. The only new thing is the else branch: when the colliding ghost is not frightened, it calls loseLife(), checks isGameOver(), and either transitions to GAME_OVER or calls resetPositions() — without ever diverging from the coordinate-comparison approach every other collision in the game already relies on.

resetPositions() vs. a Full Restart

Losing a life should not wipe the scoreboard or reset the maze — only Pac-Man and the four ghosts return to their spawn tiles, and the frightened timer is cleared so a mid-fright death doesn't carry over stale state into the next life. A full GAME_OVER restart is a deliberately heavier, separate operation.

SOURCE CODE

GameState.java

java

```
public enum GameState {  
    MENU,  
    PLAYING,  
    PAUSED,  
    GAME_OVER,  
    WIN  
}
```

GamePanel.java (excerpt — collisions & respawn)

java

```
private void checkGhostCollisions() {  
    for (Ghost ghost : ghosts) {  
        if (ghost.getRow() == pacman.getRow() && ghost.getCol() == pacman.getCol()) {  
            if (ghost.isFrightened()) {  
                scoreManager.addGhostEaten();  
                ghost.resetToSpawn();  
            } else {  
                scoreManager.loseLife();  
                if (scoreManager.isGameOver()) {  
                    state = GameState.GAME_OVER;  
                } else {  
                    resetPositions();  
                }  
            }  
            break;  
        }  
    }  
}  
  
private void resetPositions() {  
    pacman.resetToSpawn();  
    for (Ghost ghost : ghosts) ghost.resetToSpawn();  
    frightenedTimer = 0;  
}
```

GamePanel.java (excerpt — state-aware input)

java

```
if (state == GameState.GAME_OVER || state == GameState.WIN) {  
    if (code == KeyEvent.VK_ENTER) {  
        restartGame();  
    }  
    return;  
}  
  
if (state != GameState.PLAYING) return;  
switch (code) {  
    case KeyEvent.VK_UP:    pacman.setDirection(0, -1); break;  
    case KeyEvent.VK_DOWN:  pacman.setDirection(0, 1); break;  
    case KeyEvent.VK_LEFT:  pacman.setDirection(-1, 0); break;  
    case KeyEvent.VK_RIGHT: pacman.setDirection(1, 0); break;  
}
```

```
// ---- Respawn after losing a life (Week 8) ----
public void resetToSpawn() {
    row = startRow;
    col = startCol;
    x = col * Maze.TILE_SIZE;
    y = row * Maze.TILE_SIZE;
    dx = 0; dy = 0;
    nextDx = 0; nextDy = 0;
}
```

App.java, GameWindow.java, Maze.java, ScoreManager.java, RedGhost.java, BrownGhost.java, BlueGhost.java, and YellowGhost.java are unchanged from Week 7.

10-WEEK PROGRESS

Week 1 — Setup & Window	✓
Week 2 — Maze Design	✓
Week 3 — Pac-Man Character	✓
Week 4 — Dot Collection & Scoring	✓
Week 5 — Ghost Base Class	✓
Week 6 — All Four Custom Ghosts	✓
Week 7 — Power Pellets & Frightened Mode	✓
Week 8 — Collisions, Lives & Game States	✓
Weeks 9–10 — Polish + Ship	0%

UP NEXT

W
9

Polish & Audio

Add sound effects (chomp, death, siren), improve animations, and add a pause menu on top of the state machine built this week.

SoundManager.java · chomp/death/siren SFX · sprite rotation · pause overlay